

Autonomous Mobile Robot Navigation in an Environment with Humans

A ROS 2 Humble Workspace for Social-Aware Delivery Robot Navigation

Comprehensive Technical Documentation

AshwinderPalSingh

July 2026

Abstract

This document provides a comprehensive technical reference for a ROS 2 Humble workspace that implements social-aware autonomous navigation for a differential-drive delivery robot operating in a simulated Gazebo warehouse alongside four ORCA-simulated pedestrians. The **core contribution** is **social_costmap_plugin**, a custom Nav2 costmap layer written in C++ that paints a velocity-scaled, asymmetric 2-D Gaussian cost field around each detected pedestrian—stretching cost ahead of a moving person so the global planner and local controller route around their projected path rather than treating them as static point obstacles. The project includes an **A/B evaluation protocol** that holds the delivery mission, warehouse layout, and pedestrian scenario constant while toggling the social layer as the single manipulated variable, logging path length, duration, minimum pedestrian distance, and personal-space intrusion count to quantify the trade-off between navigation efficiency and social compliance.

Contents

1	Project Overview and Motivation	4
1.1	The Problem	4
1.2	The Core Contribution	4
1.3	Design Philosophy	4
2	System Architecture	6
2.1	Three-Layer Architecture	6
2.2	System Flow Diagram	7
3	Theoretical Foundations	8
3.1	The Social Costmap Model	8
3.1.1	Why an Asymmetric Gaussian	8
3.1.2	Velocity Scaling	8
3.1.3	Mathematical Formulation	8
3.1.4	The Three-Part Cost Policy	9
3.1.5	“Polite, Not Brittle” — Why Amplitude < Lethal	9
3.1.6	Affected Radius Calculation	10
3.1.7	Parameter Table	10
3.2	ORCA / RVO2 for Pedestrian Simulation	11
3.2.1	Conceptual Overview	11
3.2.2	Why ORCA Is Different from Costmap-Based Avoidance	11
3.2.3	The Robot as a Non-Avoiding ORCA Agent	11
3.2.4	The Bug That Motivated Choosing ORCA	11
3.2.5	The Tick Loop (<code>pedestrian_simulator.py</code>)	12
3.2.6	ORCA Tuning Constants	12
3.3	Nav2 Stack Fundamentals	12
3.3.1	AMCL Particle Filter Localization	13
3.3.2	NavFn Global Planner	13
3.3.3	DWB Local Controller and Critics	13
3.3.4	Costmap Layering Model	14
3.4	TF Frame Authority Design	14
4	Package-by-Package Deep Dive	16
4.1	Package 1: <code>social_nav_msgs</code>	16
4.1.1	Message Definitions	16
4.2	Package 2: <code>social_costmap_plugin</code>	16
4.2.1	Class Declaration (<code>social_layer.hpp</code>)	16

4.2.2	<code>updateCosts()</code> Logic Walkthrough	17
4.2.3	<code>updateBounds()</code> and TF Handling	18
4.2.4	Dynamic Parameter Callback	18
4.2.5	Plugin Registration	18
4.3	Package 3: <code>pedestrian_sim</code>	18
4.3.1	The <code>SimPed</code> Class	18
4.3.2	Pedestrian Configuration (<code>pedestrians.yaml</code>)	19
4.3.3	Static Obstacles in ORCA	19
4.3.4	RViz Markers	19
4.4	Package 4: <code>delivery_bot_description</code>	19
4.4.1	Robot Dimensions and Mass (from <code>delivery_bot.urdf.xacro</code>)	20
4.4.2	Gazebo Plugins and Sensors (from <code>gazebo.xacro</code>)	20
4.4.3	Inertia Macros (<code>macros.xacro</code>)	20
4.5	Package 5: <code>delivery_bot_gazebo</code>	21
4.5.1	Warehouse World (<code>warehouse.sdf</code>)	21
4.5.2	Simulation Launch (<code>sim.launch.py</code>)	21
4.6	Package 6: <code>delivery_bot_navigation</code>	21
4.6.1	<code>nav2_params.yaml</code> — Full Configuration	22
4.6.2	SLAM Configuration (<code>slam_params.yaml</code>)	22
4.6.3	Saved Map	22
4.7	Package 7: <code>delivery_bot_bringup</code>	22
4.7.1	Top-Level Launch (<code>sim_bringup.launch.py</code>)	22
4.7.2	Delivery Manager (<code>delivery_manager.py</code>)	23
4.7.3	Metrics Logger (<code>metrics_logger.py</code>)	23
5	Runtime Data Flow	24
5.1	Topic Table	24
5.2	Launch Sequence Timing	24
6	The A/B Evaluation Methodology	25
6.1	Experiment Design	25
6.2	Why Each Metric Moves	25
6.3	Intrusion Detection Logic	26
7	Key Engineering Decisions and Trade-Offs	27
8	Limitations and Future Work	29
8.1	Current Limitations (Intentional Scope Boundaries)	29
8.2	Future Work	29
9	How to Build and Run	30
9.1	Prerequisites	30
9.2	Build	30
9.3	Run Modes	30
9.3.1	Mode 1: Gazebo Simulation Only	30
9.3.2	Mode 2: SLAM / Map Building	31
9.3.3	Mode 3: Nav2 Without Pedestrians	31
9.3.4	Mode 4: Full Social Navigation	31

9.4	A/B Evaluation Protocol	31
9.4.1	Social Run (B — Layer Enabled, Default)	31
9.4.2	Baseline Run (A — Layer Disabled)	31
9.5	Live Parameter Tuning	32
A	Complete File Tree	33

Chapter 1

Project Overview and Motivation

1.1 The Problem

Standard autonomous navigation stacks treat every detected obstacle identically: a wall, a shelf, and a human being all become lethal or near-lethal cells in a 2-D occupancy grid. This works for collision avoidance—the robot does not crash—but it fails socially:

- The robot gives a walking person no more clearance than a static shelf, cutting centimetres past them if the planner sees a gap.
- The robot cannot anticipate where a person *will be* in the next few seconds; it reacts only to their current position, leading to sudden braking or tight swerves that startle pedestrians.
- A person blocking a narrow corridor is treated identically to a wall blocking it—the planner either finds a gap (passing uncomfortably close) or declares the goal unreachable and aborts.

1.2 The Core Contribution

This project extends a full Nav2 navigation stack with a single custom component: `social_costmap_plugin::SocialCostmap`, a C++ Nav2 costmap layer that subscribes to a `social_nav_msgs/Pedestrians` topic and, for each pedestrian, paints into the costmap a velocity-scaled, asymmetric 2-D Gaussian cost field whose standard deviation stretches ahead of the person proportional to their walking speed, plus a lethal-cost core over the body itself. This transforms pedestrians from invisible (or point-like) obstacles into direction-aware “expensive-but-passable” zones that cause the NavFn global planner to route around pedestrians’ projected personal space and the DWB local controller to slow and veer reactively near them.

1.3 Design Philosophy

The project deliberately constructs a **complete, measured classical baseline**. The social model is a Kirby-style proxemics Gaussian—the accepted classical starting point for social navigation research—not a learned trajectory predictor. This is an intentional scoping decision:

1. A classical baseline is reproducible and debuggable: any robot behaviour can be traced to a costmap cell value and a planner critic score.

2. The A/B evaluation protocol (Chapter 6) produces quantified metrics, not just qualitative demonstrations.
3. Learned social navigation (trajectory prediction, RL policies) is the honest *next* step, layered on and benchmarked against this validated baseline.

Chapter 2

System Architecture

The system is organized into three layers—Mission, Navigation, and Simulation—with clear topic-level boundaries between them.

2.1 Three-Layer Architecture

Layer 3: Mission and Evaluation `delivery_manager.py` drives the robot through a queue of delivery waypoints via the Nav2 Simple Commander API (`NavigateToPose` action). `metrics_logger.py` subscribes to `/odom` and `/pedestrians` and records A/B metrics (path length, duration, minimum pedestrian distance, personal-space intrusions).

Layer 2: Navigation The full Nav2 stack: AMCL particle-filter localization (or `slam_toolbox` during mapping), NavFn global planner, DWB local controller, behaviour server, and velocity smoother. The **layered costmap** is the core innovation point: static + obstacle + inflation + `social_layer` (custom plugin).

Layer 1: Simulation and Human Motion Ignition Fortress (Gazebo) runs the warehouse world and the robot’s physics and sensors. `ros_gz_bridge` bridges all needed topics. `pedestrian_sim` runs as a separate ROS 2 node (not a Gazebo actor) and publishes ground-truth pedestrian positions and velocities using ORCA collision avoidance.

2.2 System Flow Diagram

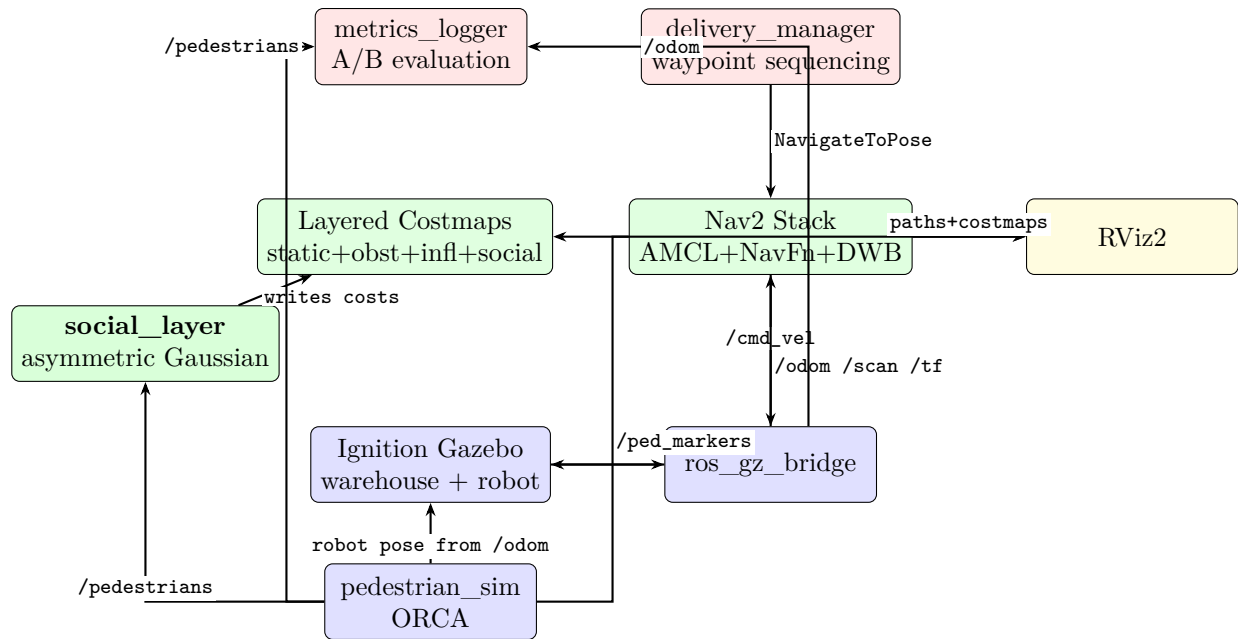


Figure 2.1: System architecture showing three layers and inter-node topic flow.

Chapter 3

Theoretical Foundations

3.1 The Social Costmap Model

3.1.1 Why an Asymmetric Gaussian

Human proxemics research (Hall, 1966; Kirby et al., 2009) shows that people require more personal space *in front of them* (where their attention and trajectory are directed) than behind or to the side. An isotropic (circular) Gaussian centred on the pedestrian’s position models only distance, not direction. An **anisotropic** Gaussian with direction-dependent standard deviations captures the observed asymmetry.

3.1.2 Velocity Scaling

A stationary person has a symmetric keep-clear zone. A walking person’s zone should stretch *ahead* proportional to speed: the faster they walk, the further ahead they will be in the next planning cycle, so the planner must give more forward clearance. This is the Kirby velocity-scaling model—it achieves anticipatory avoidance *without* explicit trajectory prediction. The velocity term encodes a first-order constant-velocity assumption directly into the cost field shape.

3.1.3 Mathematical Formulation

Directional Standard Deviations

Given a pedestrian with speed $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2}$ and heading $\theta = \text{atan2}(v_y, v_x)$, the three standard deviations are:

$$\sigma_{\text{front}} = \sigma_{\text{base}} \cdot (1 + \alpha_{\text{speed}} \cdot |\mathbf{v}|) \quad (3.1)$$

$$\sigma_{\text{back}} = \sigma_{\text{base}} \quad (3.2)$$

$$\sigma_{\text{side}} = \sigma_{\text{base}} \cdot r_{\text{side}} \quad (3.3)$$

where σ_{base} is the base personal-space radius, α_{speed} is the speed scaling factor, and r_{side} is the lateral ratio.

Standing Pedestrian (Isotropic Fallback)

When $|\mathbf{v}| < v_{\text{min}}$ (default 0.05 m/s), the pedestrian’s heading is undefined. The layer falls back to an isotropic Gaussian with $\sigma_x = \sigma_y = \sigma_{\text{base}}$ and no rotation—equal clearance in all directions.

Coordinate Rotation into the Pedestrian’s Frame

For a costmap cell at world offset $(\Delta x, \Delta y)$ from the pedestrian’s position, the offset is rotated into the pedestrian’s velocity-aligned frame:

$$l_x = \cos \theta \cdot \Delta x + \sin \theta \cdot \Delta y \quad (\text{along direction of travel}) \quad (3.4)$$

$$l_y = -\sin \theta \cdot \Delta x + \cos \theta \cdot \Delta y \quad (\text{lateral}) \quad (3.5)$$

Longitudinal Sigma Selection

The longitudinal sigma depends on whether the cell is ahead of or behind the pedestrian:

$$\sigma_x = \begin{cases} \sigma_{\text{front}} & \text{if } l_x \geq 0 \\ \sigma_{\text{back}} & \text{if } l_x < 0 \end{cases} \quad (3.6)$$

Gaussian Cost Evaluation

$$c(\Delta x, \Delta y) = A \cdot \exp\left(-\frac{1}{2} \left(\frac{l_x^2}{\sigma_x^2} + \frac{l_y^2}{\sigma_{\text{side}}^2} \right)\right) \quad (3.7)$$

where A is the amplitude (peak cost value).

3.1.4 The Three-Part Cost Policy

Each pedestrian produces three types of cost in the costmap:

1. **Lethal core.** Cells with $\sqrt{\Delta x^2 + \Delta y^2} \leq r_{\text{lethal}}$ are set to `LETHAL_OBSTACLE` (cost value 254). No path should ever route through a person’s body.
2. **Gaussian field.** Cells outside the lethal core receive cost from Equation 3.7. The amplitude A is clamped at `INSCRIBED_INFLATED_OBSTACLE` (value 253, but since the clamp uses $\min(A, 253)$ and the default $A = 220$ is already below 253, in practice the Gaussian field peaks at 220). Costs below the cutoff threshold are skipped for efficiency.
3. **Max-merge policy.** Social costs are merged into the master grid via $\max(\text{old}, \text{new})$. The social layer can only *raise* cost, never lower it. A real wall detected by lidar or loaded from the static map is never overridden by a social cost. For cells with `NO_INFORMATION`, the social cost is written directly.

3.1.5 “Polite, Not Brittle” — Why Amplitude < Lethal

Design Decision #3: Gaussian Amplitude Below Lethal

If amplitude were set to `LETHAL_OBSTACLE` (254), a pedestrian blocking the only corridor would make the goal *unreachable*—Nav2 would abort the navigation action entirely. With $A = 220$ (or even up to 252), the cost field is “expensive but passable.” If no alternative path exists, the planner produces a cautious path through the high-cost zone rather than dead-locking. The robot yields, waits, or slowly proceeds rather than declaring failure. This is the distinction between **polite** (prefers to avoid but can proceed) and **brittle** (cannot navigate at all when a person is in the way).

3.1.6 Affected Radius Calculation

The layer computes a bounding circle per pedestrian to limit the costmap iteration footprint. From `social_layer.cpp`, the affected radius is:

$$r_{\text{affected}} = \max\left(\sigma_{\text{max}} \cdot \sqrt{2 \ln \frac{\max(A, c_{\text{cutoff}} + 1)}{c_{\text{cutoff}}}}, r_{\text{lethal}}\right) + 0.10 \quad (3.8)$$

where $\sigma_{\text{max}} = \sigma_{\text{base}}(1 + \alpha_{\text{speed}} \cdot |\mathbf{v}|)$ and the 0.10 m addend is a safety margin.

3.1.7 Parameter Table

Discrepancy:

The C++ header `social_layer.hpp` declares one set of default values (amplitude=220, sigma_base=0.45, etc.), while `nav2_params.yaml` overrides them with different values for both the global and local costmaps. At runtime, the YAML values take precedence. The table below shows **both** the header defaults and the actual YAML-configured values.

Table 3.1: Social layer parameters: header defaults vs. YAML runtime values.

Parameter	HPP Default	YAML Value	Units	Meaning
amplitude	220.0	120.0	—	Peak Gaussian cost (clamped ≤ 253)
sigma_base	0.45	0.35	m	Base personal-space std. dev.
sigma_side_ratio	0.60	0.50	—	Lateral $\sigma = \text{base} \times \text{ratio}$
speed_factor	1.6	1.2	s/m	Forward stretch per m/s of speed
lethal_radius	0.30	0.20	m	Hard no-go radius over the body
cutoff	15.0	10.0	—	Ignore costs below this value
keep_time	1.0	0.6	s	Drop detections older than this
min_moving_speed	0.05	0.05*	m/s	Below this, isotropic fallback
enabled	true	true	—	Toggle layer on/off at runtime
topic	/pedestrians	/pedestrians	—	Subscription topic

*Not overridden in YAML; header default applies.

Discrepancy:

The README and PROJECT_EXPLANATION.txt list the parameter defaults as amplitude=220, sigma_base=0.45, speed_factor=1.6, lethal_radius=0.30, sigma_side_ratio=0.6, cutoff=15.0. These match the `.hpp` header defaults but **not** the actual `nav2_params.yaml` runtime configuration, which uses notably lower values (amplitude=120, sigma_base=0.35, etc.) for both costmaps. The YAML values are the ones the robot actually uses; the prose documents describe the code defaults, not the deployed tuning.

3.2 ORCA / RVO2 for Pedestrian Simulation

3.2.1 Conceptual Overview

Optimal Reciprocal Collision Avoidance (ORCA) is a multi-agent collision avoidance algorithm (van den Berg et al., 2011) that computes, for each agent in each timestep, a velocity that is **guaranteed** to be collision-free for at least a specified time horizon, given every other agent’s current position, velocity, and radius.

The key insight is that each pair of agents shares responsibility for avoiding each other equally (“reciprocal”). For each pair, ORCA constructs a half-plane constraint in velocity space—the set of velocities that keep the agent collision-free with respect to that specific neighbour. The agent’s final velocity is the point in the intersection of all half-plane constraints closest to its preferred (goal-directed) velocity, solved via linear programming.

3.2.2 Why ORCA Is Different from Costmap-Based Avoidance

ORCA and the robot’s social costmap layer are fundamentally different avoidance mechanisms:

- **ORCA:** Agent-symmetric, reciprocal, velocity-space avoidance. Each agent adjusts its velocity to split the avoidance burden. Operates per-timestep with guaranteed collision freedom.
- **Social costmap:** One-sided, configuration-space cost. The robot’s planner sees expensive cells and routes around them. Pedestrians are unaware of the robot’s planning; they move under their own ORCA model.

3.2.3 The Robot as a Non-Avoiding ORCA Agent

Design Decision: Robot in ORCA

The robot is injected into the ORCA simulation as a ground-truth-driven agent: its position and velocity are read from `/odom` every tick and written into the RVO2 simulation, but ORCA *never modifies* the robot’s motion. Pedestrians must avoid the robot; the robot never avoids pedestrians through ORCA. The robot’s own avoidance is entirely Nav2’s responsibility (via the social costmap layer). This prevents the two avoidance systems from fighting each other—if both tried to avoid simultaneously, they could produce oscillatory or contradictory trajectories.

3.2.4 The Bug That Motivated Choosing ORCA

Design Decision #2: ORCA Over Hand-Rolled Repulsion

The original pedestrian simulation used a hand-rolled repulsion force with a teleport fallback. This had a specific degenerate case: when a pedestrian walked *straight at* a stationary robot, the cross product used for side selection was near-zero (the desired travel direction was anti-parallel to the away-from-robot direction). Floating-point noise caused the side selection to oscillate randomly, and when the pedestrian got too close, the teleport heuristic fired—visually jarring and physically impossible.

ORCA’s linear-programming formulation has no “which side?” ambiguity by construction: the half-plane constraint unambiguously defines the set of collision-free velocities, and the closest-to-preferred selection is deterministic. No oscillation, no teleportation.

3.2.5 The Tick Loop (`pedestrian_simulator.py`)

At 20 Hz (configurable via `update_rate` in `pedestrians.yaml`), each tick executes:

1. **Inject robot ground truth:** Read the robot’s pose from the latest `/odom` callback. Convert body-frame twist to world-frame velocity using the quaternion yaw. Call `sim.setAgentPosition(robot_agent, ...)`, `sim.setAgentVelocity(robot_agent, ...)`, and `sim.setAgentPrefVelocity(robot_agent, ...)` (set to the robot’s actual velocity so ORCA does not try to “correct” it).
2. **Set preferred velocities:** For each pedestrian, compute the unit vector toward its current target waypoint, scale by its nominal speed, and optionally add Gaussian noise. Call `sim.setAgentPrefVelocity(agent_id, (pvx, pvy))`.
3. **ORCA step:** Call `sim.doStep()`, which solves the linear program for every agent and advances positions by $\Delta t = 1/\text{rate}$.
4. **Read back results:** For each pedestrian, read the collision-free position and velocity from ORCA via `sim.getAgentPosition()` and `sim.getAgentVelocity()`. Check waypoint arrival (tolerance 0.25 m) and advance to the next waypoint if reached.
5. **Publish:** Assemble a `Pedestrians` message (frame `odom`) and a `MarkerArray` (cylinders + velocity arrows + name labels) and publish on `/pedestrians` and `/pedestrian_markers`.

3.2.6 ORCA Tuning Constants

From `pedestrian_simulator.py`:

Table 3.2: ORCA simulation parameters.

Constant	Value	Meaning
ORCA_NEIGHBOR_DIST	4.0 m	Agent look-ahead for neighbours
ORCA_MAX_NEIGHBORS	10	Max neighbours per LP
ORCA_TIME_HORIZON	2.5 s	Planning horizon (agents)
ORCA_TIME_HORIZON_OBST	1.2 s	Planning horizon (static obstacles)
SPEED_HEADROOM	1.3	<code>maxSpeed</code> / nominal speed
ARRIVAL_TOL	0.25 m	Waypoint arrival tolerance

3.3 Nav2 Stack Fundamentals

This section covers only the Nav2 components as configured in this project’s `nav2_params.yaml`, not a generic Nav2 tutorial.

3.3.1 AMCL Particle Filter Localization

AMCL (Adaptive Monte Carlo Localization) estimates the robot’s pose within a known map by maintaining a particle cloud and updating it with laser scan observations. Its output is the `map→odom` transform—a correction that accounts for odometry drift. Key configuration from `nav2_params.yaml`:

- `base_frame_id`: `base_footprint`
- `global_frame_id`: `map`
- `odom_frame_id`: `odom`
- Particles: 500–2000
- Laser model: `likelihood_field`, max range 12.0 m, max beams 60
- Motion model: `nav2_amcl::DifferentialMotionModel`
- Auto-initialization at (0,0,0): `set_initial_pose: true`

3.3.2 NavFn Global Planner

NavFn plans a global path over the costmap using Dijkstra’s algorithm (`use_astar: false` in the YAML). It evaluates the full cost of every reachable cell, meaning the social layer’s “expensive but passable” costs directly influence path routing: the planner *prefers* lower-cost routes, bending around Gaussian cost fields even though they are not lethal.

Note:

The prose documentation (`PROJECT_EXPLANATION.txt`) mentions “A* with cost-aware planning.” However, the actual `nav2_params.yaml` sets `use_astar: false`, meaning NavFn uses Dijkstra at runtime. Both are supported by NavFn; Dijkstra explores more cells but guarantees cost-optimal paths.

3.3.3 DWB Local Controller and Critics

DWB (Dynamic Window B) was chosen over Regulated Pure Pursuit (RPP) deliberately: DWB samples trajectories against the *local costmap*, so the social layer influences local behaviour (speed, steering) in addition to the global route.

The critic list from `nav2_params.yaml` (line 136):

Table 3.3: DWB critic configuration.

Critic	Role	Scale
<code>RotateToGoal</code>	Turn to match goal orientation	32.0
<code>Oscillation</code>	Penalize oscillatory trajectories	—
<code>BaseObstacle</code>	Obstacle proximity penalty	0.02
<code>GoalAlign</code>	Align heading toward goal	24.0
<code>PathAlign</code>	Align heading with global path	32.0
<code>PathDist</code>	Minimize distance from global path	32.0
<code>GoalDist</code>	Minimize distance to goal	24.0

Discrepancy:

The prose documentation lists `ObstacleFootprint` as a DWB critic. The actual `nav2_params.yaml` uses `BaseObstacle` instead (a simpler point-based obstacle critic). It also includes `Oscillation` (not mentioned in the prose). The table above reflects the actual YAML configuration.

DWB velocity limits: $v_x^{\max} = 0.5 \text{ m/s}$, $\omega^{\max} = 1.2 \text{ rad/s}$, $a_x^{\max} = 2.0 \text{ m/s}^2$, simulation time 1.7 s, 20 vx samples $\times$ 20 vtheta samples.

3.3.4 Costmap Layering Model

Design Decision #5: Social Layer in Both Costmaps

The social layer is loaded into **both** the global and local costmaps:

- **Global costmap:** `[static_layer, obstacle_layer, social_layer, inflation_layer]`. The planner routes *around* pedestrians' projected personal space—proactive avoidance.
- **Local costmap:** `[obstacle_layer, social_layer, inflation_layer]`. DWB slows and veers reactively near pedestrians—responsive avoidance even if the global path passes close.

If the social layer were only in the global costmap, the local controller could still steer aggressively close. If only in the local costmap, the global path would cut through people and the controller would have to make large reactive corrections. Both layers together provide *plan-and-react* social compliance.

Costmap parameters (global/local):

- Global: map frame, 0.05 m resolution, update 2 Hz, `robot_radius`: 0.33 m
- Local: odom frame, rolling window 5×5 m, 0.05 m resolution, update 5 Hz, `robot_radius`: 0.33 m
- Inflation: radius 0.45 m, cost scaling factor 4.0

3.4 TF Frame Authority Design

Design Decision #6: One Transform, One Owner

Every TF transform is published by exactly one node. If two nodes publish the same transform, `tf2` arbitrates by timestamp, leading to jittering, localization jumps, and costmap corruption. This project avoids that failure mode by strict ownership assignment.

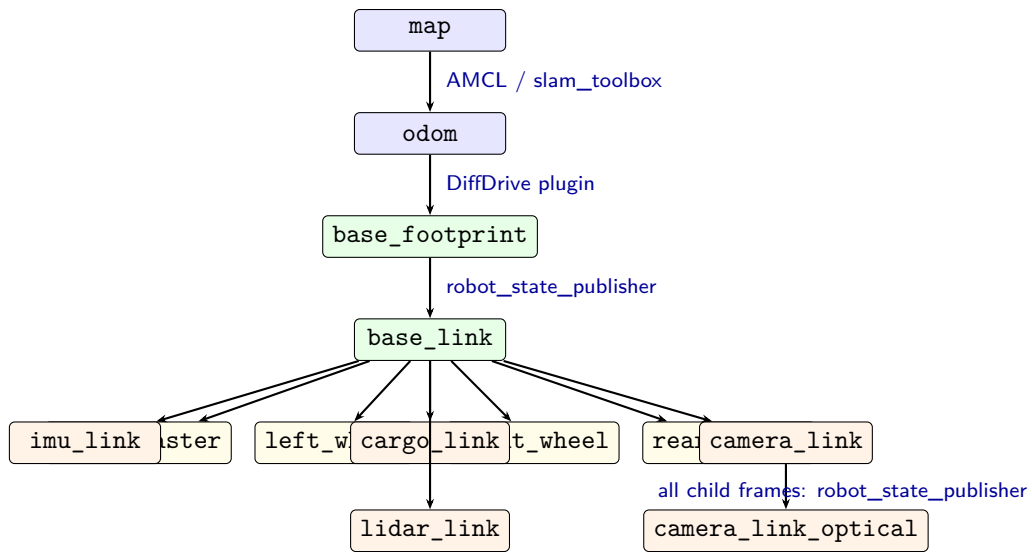


Figure 3.1: TF frame tree with publishing-node ownership annotations.

Chapter 4

Package-by-Package Deep Dive

The workspace contains seven ROS 2 packages under `src/`, presented here in dependency order.

4.1 Package 1: `social_nav_msgs`

- **Build type:** `ament_cmake` (message generation requires CMake + `rosidl`).
- **Purpose:** Defines the interface contract between any pedestrian detector (simulated or real) and the costmap plugin.

4.1.1 Message Definitions

`msg/Pedestrian.msg`:

```
1 string id                # stable track id, e.g. "ped_1"
2 geometry_msgs/Point position # [m]
3 geometry_msgs/Vector3 velocity # [m/s], world-aligned in header frame
```

`msg/Pedestrians.msg`:

```
1 std_msgs/Header header
2 social_nav_msgs/Pedestrian[] pedestrians
```

The `header.frame_id` indicates the coordinate frame of all contained positions and velocities (set to `odom` by the simulator).

4.2 Package 2: `social_costmap_plugin`

- **Build type:** `ament_cmake` (C++ shared library loaded via `pluginlib`).
- **Purpose:** The core deliverable—a Nav2 costmap layer.
- **Dependencies:** `nav2_costmap_2d`, `pluginlib`, `tf2`, `tf2_ros`, `tf2_geometry_msgs`, `geometry_msgs`, `social_nav_msgs`, `rclcpp`, `rclcpp_lifecycle`.

4.2.1 Class Declaration (`social_layer.hpp`)

```

1 namespace social_costmap_plugin {
2
3 struct TrackedPedestrian {
4     double x, y, vx, vy; // in costmap global frame
5 };
6
7 class SocialLayer : public nav2_costmap_2d::Layer {
8 public:
9     void onInitialize() override;
10    void updateBounds(double robot_x, double robot_y, double robot_yaw,
11        double* min_x, double* min_y, double* max_x, double* max_y) override;
12    void updateCosts(nav2_costmap_2d::Costmap2D& master_grid,
13        int min_i, int min_j, int max_i, int max_j) override;
14    void reset() override;
15    bool isClearable() override { return false; }
16 private:
17    void pedestriansCallback(Pedestrians::SharedPtr msg);
18    double gaussianCost(double dx, double dy,
19        double heading, double speed) const;
20    double affectedRadius(double speed) const;
21    SetParametersResult dynamicParametersCallback(
22        std::vector<rclcpp::Parameter> parameters);
23    // ... data members, parameters, mutex ...
24 };
25 } // namespace

```

Listing 4.1: SocialLayer class outline.

4.2.2 updateCosts() Logic Walkthrough

The `updateCosts()` method is the hot path, called every costmap update cycle:

1. Return immediately if disabled (`enabled_`) or no pedestrians in the transformed buffer.
2. Lock the mutex (shared with the subscriber callback).
3. For each `TrackedPedestrian`:
 - (a) Compute $\text{speed} = \sqrt{v_x^2 + v_y^2}$ and $\text{heading} = \text{atan2}(v_y, v_x)$.
 - (b) Compute the affected radius (Eq. 3.8).
 - (c) Convert the bounding circle to cell-space indices via `worldToMapEnforceBounds`, clipped to the update window $[\text{min_i}, \text{max_i}] \times [\text{min_j}, \text{max_j}]$.
 - (d) For each cell (i, j) in the bounding box:
 - Convert to world coordinates via `mapToWorld`.
 - Compute $\Delta x = w_x - p_x$, $\Delta y = w_y - p_y$.
 - If $\sqrt{\Delta x^2 + \Delta y^2} \leq r_{\text{lethal}}$: set cost to `LETHAL_OBSTACLE` (254).
 - Else: evaluate `gaussianCost()`. If below cutoff, skip. Otherwise clamp to $\leq \text{INSCRIBED_INFLATED_COST}$ and cast to `unsigned char`.
 - Merge: if old cost is `NO_INFORMATION`, write directly; otherwise `std::max(old, new)`.

4.2.3 updateBounds() and TF Handling

Each update cycle, `updateBounds()` transforms pedestrian detections from their source frame (the `header.frame_id` of the `Pedestrians` message, typically `odom`) into the costmap's global frame (`map` for the global costmap, `odom` for the local costmap) via `tf2`.

- **Position:** Transformed as a `PointStamped` (full 3-D transform).
- **Velocity:** Transformed as a `Vector3Stamped` (rotation only—velocity is a free vector, not a position).
- **Staleness check:** If the message timestamp is older than `keep_time` (default 1.0s), the entire message is dropped. Stale detections are not frozen in place.
- **current_ always true:** Even when the layer has no pedestrians or is disabled, it reports `current_ = true`. Failing to do this makes Nav2 treat the entire costmap as stale—a non-obvious Nav2 gotcha.

4.2.4 Dynamic Parameter Callback

All parameters can be changed at runtime via `ros2 param set`. The callback clamps values to safe ranges: `sigma_base` ≥ 0.05 , `cutoff` ≥ 1.0 , `keep_time` ≥ 0.1 , etc.

4.2.5 Plugin Registration

`social_costmap_plugin.xml`:

```
1 <class name="social_costmap_plugin/SocialLayer"
2     type="social_costmap_plugin::SocialLayer"
3     base_class_type="nav2_costmap_2d::Layer">
```

Registered via `PLUGINLIB_EXPORT_CLASS` at the bottom of `social_layer.cpp` and `pluginlib_export_plugin` in `CMakeLists.txt`.

4.3 Package 3: pedestrian_sim

- **Build type:** `ament_python` (pure Python node).
- **External dependency:** Python-RVO2 (built from source; not available via `rosdep`).
- **Entry point:** `pedestrian_simulator` $\rightarrow$ `pedestrian_sim.pedestrian_simulator:main`.

4.3.1 The SimPed Class

Tracks one pedestrian's waypoint progress. Fields: `name`, `waypoints` (list of (x, y) tuples), `speed`, `radius`, `mode` (`pingpong` or `loop`), `noise_std`, current position (x, y) , velocity (v_x, v_y) , `target_idx`, `direction`.

Methods:

- `preferred_velocity()`: Unit vector toward current waypoint, scaled by speed, with optional Gaussian noise.

- `advance_if_arrived()`: If within `ARRIVAL_TOL` (0.25 m), advance to next waypoint.
- `_advance()`: Handle `loop` (modular index) or `pingpong` (reverse direction at endpoints).

4.3.2 Pedestrian Configuration (`pedestrians.yaml`)

Table 4.1: Configured pedestrian scenarios.

ID	Path	Speed	Radius	Mode	Waypoints
ped_1	N corridor E–W	0.55 m/s	0.30 m [*]	pingpong	(3.5, 3.5) $\leftrightarrow$ (−4.5, 3.5)
ped_2	S corridor E–W	0.60 m/s	0.30 m [*]	pingpong	(−4.5, −3.5) $\leftrightarrow$ (3.5, −3.5)
ped_3	E side N–S	0.50 m/s	0.30 m [*]	pingpong	(3.5, −3.0) $\leftrightarrow$ (3.5, 3.0)
ped_4	W side loop	0.45 m/s	0.30 m [*]	loop	4 waypoints forming a rectangle

^{*}Radius not specified in YAML; uses the code default of 0.30 m declared in `pedestrian_simulator.py` line 178.

4.3.3 Static Obstacles in ORCA

The pedestrian simulator registers warehouse obstacles as ORCA static obstacles using their exact SDF footprints (verified against `warehouse.sdf`):

Table 4.2: Static obstacles registered in ORCA (from `STATIC_OBSTACLES`).

Name	cx	cy	half_x	half_y
shelf_a	−1.0	2.0	1.25	0.30
shelf_b	−1.0	−2.0	1.25	0.30
box_1	4.8	3.2	0.40	0.40
box_2	4.8	−3.2	0.40	0.40
pillar	−3.5	−2.5	0.20	0.20
wall_north	0.0	4.5	6.15	0.075
wall_south	0.0	−4.5	6.15	0.075
wall_east	6.0	0.0	0.075	4.65
wall_west	−6.0	0.0	0.075	4.65

4.3.4 RViz Markers

For each pedestrian, three markers are published per tick:

1. **Cylinder** (body): Height 1.7 m, diameter $2 \times$ radius, colour-coded per pedestrian.
2. **Arrow** (velocity): Shaft from position to position $+(v_x, v_y)$, shown only when speed > 0.02 m/s.
3. **Text** (label): Name floating at $z = 2.0$ m.

All markers have a 0.5 s lifetime for automatic cleanup.

4.4 Package 4: `delivery_bot_description`

- **Build type**: `ament_cmake` (installs URDF + launch).
- **Purpose**: Robot physical model and sensor definitions.

4.4.1 Robot Dimensions and Mass (from `delivery_bot.urdf.xacro`)

Table 4.3: Robot link dimensions and masses.

Link	Geometry	Mass	Dimensions / Notes
base_footprint	(virtual)	—	Ground-plane frame at (0, 0, 0)
base_link	Box	12.0 kg	$0.50 \times 0.40 \times 0.20$ m, $z = 0.14$ m
left/right_wheel	Cylinder	1.0 kg ea.	$r = 0.09$ m, $w = 0.04$ m, sep. 0.44 m
front/rear_caster	Sphere	0.4 kg ea.	$r = 0.045$ m, $x = \pm 0.20$ m
cargo_link	Box	3.0 kg	$0.34 \times 0.34 \times 0.22$ m
lidar_link	Cylinder	0.2 kg	$r = 0.045$ m, $h = 0.05$ m
camera_link	Box	0.1 kg	$0.03 \times 0.09 \times 0.03$ m
camera_link_optical	(virtual)	—	Rotated -90° roll, -90° yaw
imu_link	Box	0.05 kg	$0.02 \times 0.02 \times 0.02$ m

Discrepancy:

`PROJECT_EXPLANATION.txt` refers to the camera optical frame as `camera_optical_link`. The actual URDF names it `camera_link_optical`. The URDF name is authoritative.

4.4.2 Gazebo Plugins and Sensors (from `gazebo.xacro`)

Table 4.4: Ignition Fortress plugins and sensor configurations.

Plugin/Sensor	Publishes	Subscribes	Rate
DiffDrive	/odom, /tf	/cmd_vel	50 Hz
JointStatePublisher	/joint_states	—	—
GPU Lidar	/scan	—	10 Hz
RGBD Camera	/camera/*	—	10 Hz
IMU	/imu	—	100 Hz

Discrepancy:

`PROJECT_EXPLANATION.txt` states the lidar runs at **30 Hz** and the IMU at **50 Hz**. The actual `gazebo.xacro` specifies the lidar `update_rate` as **10** and the IMU as **100**. The URDF/Xacro values are authoritative.

Lidar specifications: 360 horizontal samples, $\pm\pi$ scan angle, range 0.15–12.0 m, 0.01 m range resolution.

Camera: 320×240 px, RGB8 format, 1.25 rad horizontal FOV, clip 0.2–8.0 m.

Caster friction: $\mu_1 = \mu_2 = 0.001$ (near-frictionless for differential-drive kinematics). Drive wheel friction: $\mu_1 = \mu_2 = 1.2$.

4.4.3 Inertia Macros (`macros.xacro`)

Standard solid-body inertia formulas for box ($I_{xx} = \frac{1}{12}m(y^2 + z^2)$), cylinder ($I_{xx} = \frac{1}{12}m(3r^2 + l^2)$), and sphere ($I_{xx} = \frac{2}{5}mr^2$).

4.5 Package 5: delivery_bot_gazebo

- **Build type:** `ament_cmake` (installs worlds + launch).
- **Purpose:** Warehouse world definition and simulation bringup.

4.5.1 Warehouse World (`warehouse.sdf`)

Table 4.5: Warehouse static obstacles (from `warehouse.sdf`).

Model	Pose (x, y)	Size (x, y, z)	Material
wall_north	(0, 4.5)	$12.3 \times 0.15 \times 1.0$ m	cream
wall_south	(0, -4.5)	$12.3 \times 0.15 \times 1.0$ m	cream
wall_east	(6, 0)	$0.15 \times 9.3 \times 1.0$ m	cream
wall_west	(-6, 0)	$0.15 \times 9.3 \times 1.0$ m	cream
shelf_a	(-1, 2)	$2.5 \times 0.6 \times 0.8$ m	brown
shelf_b	(-1, -2)	$2.5 \times 0.6 \times 0.8$ m	brown
box_1	(4.8, 3.2)	$0.8 \times 0.8 \times 0.6$ m	tan
box_2	(4.8, -3.2)	$0.8 \times 0.8 \times 0.6$ m	tan
pillar	(-3.5, -2.5)	$0.4 \times 0.4 \times 1.0$ m	grey

Interior dimensions: approximately 12×9 m (wall inner faces at $x \approx \pm 5.925$, $y \approx \pm 4.425$). Physics: 4 ms step size, DART (default; SDF `type="ignored"`), real-time factor 1.0. Required system plugins: Physics, UserCommands, SceneBroadcaster, Sensors (ogre2 renderer), Imu.

4.5.2 Simulation Launch (`sim.launch.py`)

Starts (in order):

1. Ignition Gazebo with `-r` (run immediately) and the warehouse world.
2. `robot_state_publisher` via `description.launch.py`.
3. Robot spawn from `/robot_description` at (0,0,0.20).
4. `ros_gz_bridge` bridging all topics (see Table 5.1).

The robot spawn $z = 0.20$ m drops the robot slightly above ground so physics settles it onto the floor plane.

4.6 Package 6: delivery_bot_navigation

- **Build type:** `ament_cmake` (installs config, launch, maps, rviz).
- **Purpose:** All Nav2 and SLAM configuration.

4.6.1 nav2_params.yaml — Full Configuration

This 322-line YAML configures every Nav2 component. Key sections (selected highlights—see Section 3.3 for the navigation components and Table 3.1 for social layer parameters):

- **AMCL:** 500–2000 particles, likelihood field laser model, auto-init at (0, 0, 0).
- **Controller server:** DWB at 20 Hz, 7 critics (Table 3.3).
- **Planner server:** NavFn with Dijkstra (`use_astar: false`), tolerance 0.5 m.
- **Behaviour server:** spin, backup, drive_on_heading, wait.
- **Velocity smoother:** 20 Hz, open-loop feedback, max velocity [0.5, 0.0, 1.2].
- **Waypoint follower:** 20 Hz loop rate, wait-at-waypoint plugin (200 ms pause).

4.6.2 SLAM Configuration (slam_params.yaml)

78-line `slam_toolbox` async online mapping configuration:

- **Solver:** Ceres (SPARSE_NORMAL_CHOLESKY, Levenberg-Marquardt).
- **Frames:** map, odom, base_footprint.
- **Resolution:** 0.05 m. Max laser range: 12.0 m.
- **Loop closure enabled.**

4.6.3 Saved Map

`maps/warehouse.yaml` + `warehouse.pgm`: the occupancy grid generated by driving the robot with teleop + `slam_toolbox` (Phase 3).

4.7 Package 7: delivery_bot_bringup

- **Build type:** ament_python.
- **Entry points:** `delivery_manager`, `metrics_logger`.
- **Purpose:** Top-level orchestration, mission management, and A/B evaluation logging.

4.7.1 Top-Level Launch (sim_bringup.launch.py)

Arguments: `mode` (nav|slam), `slam` (True|False), `pedestrians` (True|False), `rviz` (True|False).

Staged launch sequence:

1. $t = 0$ s: `sim.launch.py` (Gazebo + bridge).
2. $t = 5$ s: `nav.launch.py` or `slam.launch.py` (depending on mode).
3. $t = 6$ s: `pedestrians.launch.py` (if enabled).
4. $t = 7$ s: RViz2 (if enabled).

The staggered timing avoids Gazebo startup race conditions: Nav2 needs `/scan` and `/tf` to be publishing before it can initialize; the pedestrian simulator needs `/odom`; RViz needs costmap topics.

4.7.2 Delivery Manager (`delivery_manager.py`)

Uses `nav2_simple_commander.BasicNavigator` to send sequential `NavigateToPose` goals. Configuration from `mission.yaml`:

Table 4.6: Mission configuration.

Parameter	Value
<code>localizer</code>	<code>amcl</code>
<code>loops</code>	2
<code>stop_on_failure</code>	false
<code>leg_timeout</code>	180.0s
<code>set_initial_pose</code>	false
<code>waypoints</code>	4 stops (see below)

Waypoints (map frame): NW pickup $(-4.5, 3.5, \pi) \rightarrow$ NE dropoff $(2.0, 3.5, 0) \rightarrow$ SE dropoff $(3.5, -3.5, -\pi/2) \rightarrow$ SW return $(-4.5, -3.5, \pi)$. These cross all four pedestrian paths.

Each leg has a timeout guard: if `navigation_time` exceeds `leg_timeout`, the leg is cancelled and reported as `TIMED_OUT`. The final mission report counts succeeded/failed/ cancelled/timed-out legs and total navigation time.

4.7.3 Metrics Logger (`metrics_logger.py`)

Subscribes to `/odom` (at QoS depth 20) and `/pedestrians` (at QoS depth 20). Samples at 10 Hz (default).

Per-sample computation:

1. Robot position from latest `/odom`.
2. For each pedestrian, compute Euclidean distance to robot.
3. Record minimum distance and closest pedestrian ID.
4. If min distance < 0.8 m (`personal_space` parameter), increment intrusion counter.

Path length: Integrated from `/odom` at full rate (not just at sample points), with a jump filter: steps > 0.5 m between consecutive 50 Hz odom samples are rejected as teleport glitches.

Stale detection check: If no `/pedestrians` message has arrived within `detection_timeout` (1.0s wall time), sampling pauses.

Frame consistency: A one-time warning is printed if the pedestrians message `frame_id` differs from the odom frame—distances assume a shared frame and may be biased by localization drift if frames differ.

CSV output schema:

```
1 t_s, robot_x, robot_y, min_ped_dist_m, closest_ped, path_length_m
```

File naming: `social_metrics_<label>_<timestamp>.csv` in `/tmp` (default `output_dir`).

Chapter 5

Runtime Data Flow

5.1 Topic Table

Table 5.1: ROS 2 topics (verified against source code).

Topic	Type	Publisher	Subscriber(s)
/clock	rosgraph_msgs/Clock	Gazebo (bridged)	All nodes (sim time)
/cmd_vel	geometry_msgs/Twist	Nav2 vel. smoother	Gazebo DiffDrive
/odom	nav_msgs/Odometry	Gazebo DiffDrive	Nav2, ped. sim, metrics
/tf	tf2_msgs/TFMessage	DiffDrive, AMCL, RSP	Nav2, RViz
/scan	sensor_msgs/LaserScan	Gazebo GPU lidar	Nav2 costmap
/imu	sensor_msgs/Imu	Gazebo IMU	(available)
/joint_states	sensor_msgs/JointState	Gazebo (bridged)	robot_state_pub.
/pedestrians	social_nav_msgs/Pedestrian	pedestrian_sim	social_layer, metrics
/pedestrian_markers	visualization_msgs/MarkerArray	pedestrian_sim	RViz
/map	nav_msgs/OccupancyGrid	map_server / SLAM	Nav2
/camera/*	sensor_msgs/*	Gazebo RGBD	(available)

5.2 Launch Sequence Timing

The staged timing in `sim_bringup.launch.py` exists to avoid race conditions:

1. $t = 0$: Gazebo starts, spawns the robot, starts the bridge. The bridge needs a few seconds to establish Ignition transport subscriptions and begin forwarding `/clock`, `/scan`, `/odom`, `/tf`.
2. $t = 5$ s: Nav2 starts. It immediately needs `/scan` and `/tf` to initialize AMCL and the costmaps. Starting before the bridge is ready causes transform timeouts and costmap initialization failures.
3. $t = 6$ s: Pedestrian simulator starts. It needs `/odom` to inject the robot into ORCA. Starting before `/odom` publishes would leave the robot at (0,0) with zero velocity in ORCA.
4. $t = 7$ s: RViz starts. It needs costmap and path topics, which Nav2 only publishes after its lifecycle nodes reach the `active` state.

Chapter 6

The A/B Evaluation Methodology

6.1 Experiment Design

The A/B evaluation is a controlled experiment with one manipulated variable:

Table 6.1: A/B experiment design.

Variable	Baseline (A)	Social (B)
Social layer	Disabled	Enabled (default)
Delivery mission	Identical	Identical
Warehouse world	Identical	Identical
Pedestrian scenario	Identical	Identical
Nav2 configuration	Identical	Identical

6.2 Why Each Metric Moves

Table 6.2: Expected metric changes and mechanistic explanations.

Metric	Baseline	Social	Mechanism
Min ped. distance	Lower	Higher	Social costs create keep-clear zones; planner routes around them.
Intrusions (< 0.8 m)	More	Fewer	With layer off, pedestrians are invisible to Nav2—the robot drives through them.
Path length	Shorter	Longer	Global planner detours around non-lethal but expensive Gaussian cells.
Duration	Shorter	Longer	Longer paths + DWB slowing near high-cost cells increases travel time.

The trade-off is **intentional**: the social configuration accepts a modest route/time cost (a few percent more path length) to buy a large reduction in personal-space violations—fewer intrusions and greater minimum pedestrian distance.

6.3 Intrusion Detection Logic

From `metrics_logger.py` (line 149):

```
1 if min_dist < self.personal_space: # default 0.8 m
2     self.intrusions += 1
```

An *intrusion event* is a per-sample binary: at each 10 Hz sample, if the minimum Euclidean distance to any pedestrian (computed in the odom frame) is below the `personal_space` threshold (default 0.8 m), the sample counts as an intrusion. The total count and percentage of all samples are reported.

Chapter 7

Key Engineering Decisions and Trade-Offs

Decision 1: Pedestrians as a ROS Node, Not Gazebo Actors

Decision: Simulate pedestrians as a pure ROS 2 node publishing ground-truth positions, not as Gazebo actors with physics.

Alternatives considered: Gazebo actor scripting; `gazebo_ros` pose teleport.

Rationale:

1. Ignition Fortress actors have no collision physics.
2. Clean separation of concerns: the project’s contribution is the *navigation-side* integration. Perception is simulated as ground truth and can be swapped for a real detector later without touching the costmap plugin—the interface is the `Pedestrians` message.
3. The A/B demo is unambiguous: with the social layer off, the baseline stack *cannot see the pedestrians at all* (they are not lidar obstacles), making it the starkest possible evidence that the social layer—and nothing else—produces the avoidance.

What breaks under the alternative: Gazebo actors would require perception (leg detection, vision tracking), entangling the contribution with a different, large problem and making the A/B comparison ambiguous.

Decision 2: ORCA (RVO2) for Pedestrian Avoidance

Decision: Use the RVO2 library’s ORCA implementation for pedestrian collision avoidance.

Alternative: Hand-rolled repulsion force with teleport fallback.

Rationale: The repulsion force had a specific bug: when a pedestrian walked straight at a stopped robot, the cross product for side selection was near-zero (floating-point noise), causing oscillation and then teleportation. ORCA’s LP formulation guarantees a collision-free velocity each step with no “which side?” ambiguity.

What breaks under the alternative: Pedestrians oscillate and teleport in the degenerate head-on case, producing unreliable simulation.

Decision 3: Gaussian Amplitude Below Lethal (253)

Decision: Cap the Gaussian cost amplitude below `LETHAL_OBSTACLE` (254).

Alternative: Set amplitude to lethal for hard avoidance.

Rationale: If amplitude equals lethal, a pedestrian blocking the only corridor makes the goal unreachable—Nav2 aborts. With amplitude below lethal (YAML: 120, code default: 220), the cost is “expensive but passable.” The robot yields, waits, or plans through slowly rather than dead-locking.

What breaks under the alternative: Navigation failure (goal unreachable) whenever a pedestrian blocks a bottleneck.

Decision 4: `frame_id` Is `odom`, Not `map`

Decision: The pedestrian simulator publishes detections in the `odom` frame.

Alternative: Publish in the `map` frame.

Rationale: The simulator publishes raw world/Gazebo coordinates. The robot spawns at the origin, so `odom` coordinates equal world coordinates. If `frame_id` were `map`, pedestrian positions would drift whenever AMCL applies a `map`→`odom` correction (e.g., after a “2D Pose Estimate” in RViz).

What breaks under the alternative: Pedestrian positions jump every time AMCL updates its localization correction.

Decision 5: Social Layer in Both Costmaps

Decision: Wire the social layer into both the global and local costmaps.

Alternative: Social layer in only one costmap.

Rationale: Global costmap → proactive planning (NavFn routes around pedestrians). Local costmap → reactive control (DWB slows/veers near pedestrians). Both together provide plan-and-react social compliance.

What breaks under the alternative: Global-only: the local controller ignores pedestrians and steers aggressively close. Local-only: the global path cuts through people, forcing large reactive corrections.

Decision 6: TF Authority — One Transform, One Owner

Decision: Strict ownership: each TF transform is published by exactly one node.

Assignment: `map`→`odom`: AMCL or `slam_toolbox` (never both); `odom`→`base_footprint`: Gazebo DiffDrive; `base_footprint`→everything else: `robot_state_publisher`.

What breaks under the alternative: Duplicate publishers cause `tf2` timestamp arbitration, leading to jittering, localization jumps, and costmap corruption.

Chapter 8

Limitations and Future Work

8.1 Current Limitations (Intentional Scope Boundaries)

These are deliberate scoping decisions, not flaws—each isolates the project’s navigation-side contribution from orthogonal subproblems:

1. **Perception is ground truth.** The pedestrian simulator publishes exact positions and velocities. No real detection or tracking pipeline exists. This isolates the costmap layer’s contribution from perception noise and allows clean A/B evaluation.
2. **Constant-velocity assumption.** The cost field is shaped by *current* velocity; there is no multi-step trajectory prediction. The velocity-scaled sigma is a first-order approximation of anticipatory avoidance.
3. **Single robot.** The workspace demonstrates one robot plus a delivery mission manager. Multi-robot operation would require namespaced Nav2 stacks and an inter-robot cost channel.
4. **Classical social model, not SOTA.** The Kirby Gaussian model is the accepted classical baseline for social navigation. Current state-of-the-art uses learned trajectory prediction. Framing this honestly is stronger than overclaiming.

8.2 Future Work

1. Replace ground-truth pedestrian messages with real detection and tracking (leg detector, vision-based tracker) publishing the same `Pedestrians` message interface.
2. Add learned trajectory prediction (e.g., Social-LSTM, Trajectron++) as input to the social costmap—per-pedestrian predicted waypoints feeding directional cost.
3. Add automated experiment scripts for repeated A/B trials with statistical analysis.
4. Add plots generated from metrics CSV files (distance-over-time, path visualizations).
5. Extend the launch setup for multiple robots with namespaced Nav2 stacks and inter-robot costmap sharing.
6. Add CI for message generation, plugin compilation, and Python linting.

Chapter 9

How to Build and Run

9.1 Prerequisites

```
1 sudo apt update
2 sudo apt install -y \
3   ros-humble-desktop ros-humble-ros-gz \
4   ros-humble-navigation2 ros-humble-nav2-bringup \
5   ros-humble-slam-toolbox ros-humble-nav2-simple-commander \
6   ros-humble-teleop-twist-keyboard ros-humble-xacro \
7   ros-humble-joint-state-publisher-gui \
8   python3-colcon-common-extensions python3-rosdep \
9   cmake build-essential
```

Listing 9.1: System dependencies (Ubuntu 22.04).

```
1 pip3 install Cython
2 git clone https://github.com/sybrenstuvel/Python-RV02.git
3 cd Python-RV02 && pip3 install . && cd ..
4 python3 -c "import rvo2" # should print nothing
```

Listing 9.2: Python-RV02 (ORCA) installation.

9.2 Build

```
1 cd Autonomous-mobile-robot-navigation-in-enviroment-with-humans
2 rosdep update
3 rosdep install --from-paths src --ignore-src -r -y
4 colcon build --symlink-install
5 source install/setup.bash
```

Listing 9.3: Workspace build.

9.3 Run Modes

9.3.1 Mode 1: Gazebo Simulation Only

```

1 ros2 launch delivery_bot_gazebo sim.launch.py
2 # In another terminal:
3 ros2 run teleop_twist_keyboard teleop_twist_keyboard

```

9.3.2 Mode 2: SLAM / Map Building

```

1 ros2 launch delivery_bot_bringup sim_bringup.launch.py \
2   mode:=slam rviz:=True
3 # Drive around, then save:
4 ros2 run nav2_map_server map_saver_cli \
5   -f src/delivery_bot_navigation/maps/warehouse
6 colcon build --packages-select delivery_bot_navigation \
7   --symlink-install
8 source install/setup.bash

```

9.3.3 Mode 3: Nav2 Without Pedestrians

```

1 ros2 launch delivery_bot_bringup sim_bringup.launch.py \
2   mode:=nav pedestrians:=False
3 # Use RViz "2D Goal Pose" to test navigation.

```

9.3.4 Mode 4: Full Social Navigation

```

1 ros2 launch delivery_bot_bringup sim_bringup.launch.py \
2   mode:=nav

```

9.4 A/B Evaluation Protocol

9.4.1 Social Run (B — Layer Enabled, Default)

```

1 ros2 run delivery_bot_bringup metrics_logger --ros-args \
2   -p label:=social -p use_sim_time:=true

```

Listing 9.4: Terminal 1: Metrics logger.

```

1 ros2 run delivery_bot_bringup delivery_manager --ros-args \
2   --params-file install/delivery_bot_bringup/share/\
3   delivery_bot_bringup/config/mission.yaml

```

Listing 9.5: Terminal 2: Delivery mission.

Stop `metrics_logger` with Ctrl-C after the mission report prints.

9.4.2 Baseline Run (A — Layer Disabled)

```
1 ros2 param set /global_costmap/global_costmap \  
2   social_layer.enabled false  
3 ros2 param set /local_costmap/local_costmap \  
4   social_layer.enabled false
```

Listing 9.6: Disable the social layer at runtime.

Then repeat the metrics logger (with `label:=baseline`) and delivery manager commands.

9.5 Live Parameter Tuning

```
1 ros2 param set /global_costmap/global_costmap \  
2   social_layer.amplitude 180.0  
3 ros2 param set /global_costmap/global_costmap \  
4   social_layer.speed_factor 2.2  
5 ros2 param set /global_costmap/global_costmap \  
6   social_layer.sigma_base 0.5  
7 # Apply the same to /local_costmap/local_costmap
```

Appendix A

Complete File Tree

Verified against the actual repository (all 49 files under `src/`):

```
social_nav_ws/  
+-- README.md  
+-- PROJECT_EXPLANATION.txt  
+-- docs/  
|   +-- DESIGN.md  
|   +-- images/ (simulation.gif, gazebo-warehouse.png.jpeg,  
|               rviz-navigation.png.jpeg, rviz-social-costmap.png.jpeg,  
|               A--B.gif)  
+-- urdf_check.rviz  
+-- src/  
    +-- social_nav_msgs/  
    |   +-- CMakeLists.txt, package.xml  
    |   +-- msg/Pedestrian.msg, Pedestrians.msg  
    +-- social_costmap_plugin/  
    |   +-- CMakeLists.txt, package.xml, social_costmap_plugin.xml  
    |   +-- include/social_costmap_plugin/social_layer.hpp  
    |   +-- src/social_layer.cpp  
    +-- pedestrian_sim/  
    |   +-- package.xml, setup.py, setup.cfg, resource/pedestrian_sim  
    |   +-- config/pedestrians.yaml  
    |   +-- launch/pedestrians.launch.py  
    |   +-- pedestrian_sim/__init__.py, pedestrian_simulator.py  
    +-- delivery_bot_description/  
    |   +-- CMakeLists.txt, package.xml  
    |   +-- meshes/.gitkeep  
    |   +-- launch/description.launch.py  
    |   +-- urdf/delivery_bot.urdf.xacro, gazebo.xacro, macros.xacro  
    +-- delivery_bot_gazebo/  
    |   +-- CMakeLists.txt, package.xml  
    |   +-- worlds/warehouse.sdf  
    |   +-- launch/sim.launch.py  
    +-- delivery_bot_navigation/  
    |   +-- CMakeLists.txt, package.xml  
    |   +-- config/nav2_params.yaml, slam_params.yaml  
    |   +-- maps/warehouse.yaml, warehouse.pgm, .gitkeep, README.md  
    |   +-- rviz/nav.rviz  
    |   +-- launch/nav.launch.py, slam.launch.py  
    +-- delivery_bot_bringup/  
    |   +-- package.xml, setup.py, setup.cfg  
    |   +-- resource/delivery_bot_bringup  
    |   +-- config/mission.yaml
```

```
+-- launch/sim_bringup.launch.py  
+-- delivery_bot_bringup/  
    +-- __init__.py, delivery_manager.py, metrics_logger.py
```

Note:

All files listed in `PROJECT_EXPLANATION.txt` Section 7 exist in the repository. No files described in the prose are missing, and no significant undocumented files were found (only standard build artifacts like `.gitkeep`, `setup.cfg`, and `resource/` marker files).